

Assignment 2: N-gram Language Modeling and OPT-1.3B Experiments

Jose Alvarez
NLP 201: Natural Language Processing I

1 Introduction

This report describes my implementation and evaluation of n -gram language models and a small set of experiments with the OPT-1.3B large language model.

For Parts 1 and 2, I implemented unigram, bigram, and trigram language models using maximum likelihood estimation (MLE), explored linear interpolation between these models, and reported perplexity on the train, development, and test splits of the One Billion Word Benchmark subset provided in the assignment.

For Part 3, I replaced GPT-3 with the open-source OPT-1.3B model and evaluated it on zero-shot and few-shot reading comprehension, as well as a simple summarization task, using only prompt-based (in-context) learning.

2 Data and Preprocessing

Each line in the three files `1b_benchmark.train.tokens`, `1b_benchmark.dev.tokens`, and `1b_benchmark.test.tokens` is already whitespace-tokenized, so tokenization simply uses `str.split()`.

2.1 Handling Rare Words

Following the instructions, I mapped tokens in the *training* split that occurred fewer than 3 times to a special `<UNK>` token. After this preprocessing, the final vocabulary (including `<UNK>` and `<STOP>`, but excluding `<START>`) contained **26,602** unique types.

For the dev and test splits, any token not seen in the training vocabulary was also mapped to `<UNK>`.

2.2 Sentence Boundary Markers

For n -gram modeling, I augmented each sentence with boundary markers:

- Two `<START>` tokens were prepended.
- One `<STOP>` token was appended.

This yields sequences of the form:

$$[\text{<START>}, \text{<START>}, w_1, \dots, w_T, \text{<STOP>}].$$

As required, in the trigram model the first non-`<START>` token uses the bigram probability $p(w \mid \text{<START>})$.

3 Maximum Likelihood n -gram Models (Part 1)

3.1 MLE Parameter Estimation

I implemented a single `NgramLM` class that stores:

- Unigram counts $c(w)$,
- Bigram counts $c(w_{i-1}, w_i)$,
- Trigram counts $c(w_{i-2}, w_{i-1}, w_i)$,

along with context counts $c(w_{i-1})$ and $c(w_{i-2}, w_{i-1})$.

MLE probabilities are:

$$\begin{aligned} p_{\text{uni}}(w) &= \frac{c(w)}{\sum_{w'} c(w')}, \\ p_{\text{bi}}(w_i \mid w_{i-1}) &= \begin{cases} \frac{c(w_{i-1}, w_i)}{c(w_{i-1})} & \text{if } c(w_{i-1}) > 0, \\ 0 & \text{otherwise,} \end{cases} \\ p_{\text{tri}}(w_i \mid w_{i-2}, w_{i-1}) &= \begin{cases} \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})} & \text{if } c(w_{i-2}, w_{i-1}) > 0, \\ 0 & \text{otherwise.} \end{cases} \end{aligned}$$

3.2 Perplexity Computation

For a tokenized sentence, I compute the log probability under a given model and count M , the total number of tokens excluding any `<START>` but including `<STOP>`. Perplexity over a set of sentences is:

$$\text{PPL} = \exp \left(-\frac{1}{M} \sum_{i=1}^M \log p(w_i \mid \cdot) \right).$$

To avoid numerical issues when an MLE probability is exactly zero (due to unseen n -grams), I clamp p to a small floor of 10^{-12} before taking the logarithm. This does not change the overall trend but prevents NaN values.

3.3 Results

Table 1 shows the MLE perplexities for the unigram, bigram, and trigram models on train, dev, and test.

Model	Train PPL	Dev PPL	Test PPL
Unigram	976.54	892.25	896.50
Bigram	77.07	8995.04	8973.07
Trigram	7.87	4.02×10^7	3.90×10^7

Table 1: MLE n -gram perplexities on train, dev, and test.

On the training set, the expected pattern holds: higher-order n -grams (bigrams and especially trigrams) achieve substantially lower perplexity by exploiting more context. However, on dev and

test, the bigram and trigram models suffer from severe data sparsity: many test contexts never appeared in training, leading to tiny clamped probabilities and extremely large perplexities. This demonstrates classic overfitting behavior of MLE n -gram models without smoothing.

The unigram model, although much less expressive, generalizes more smoothly across splits because it does not condition on longer contexts.

4 Linear Interpolation Smoothing (Part 2)

4.1 Interpolated Model

To reduce sparsity and improve generalization, I implemented linear interpolation between the MLE unigram, bigram, and trigram models:

$$\theta'(w_i \mid w_{i-2}, w_{i-1}) = \lambda_1 p_{\text{uni}}(w_i) + \lambda_2 p_{\text{bi}}(w_i \mid w_{i-1}) + \lambda_3 p_{\text{tri}}(w_i \mid w_{i-2}, w_{i-1}),$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$ and each $\lambda_i \geq 0$.

The interpolated sentence probability is obtained by replacing the MLE trigram probability with θ' at each position (with the same special handling of the first non-**<START>** token).

4.2 Hyperparameter Search

I used the development set to select interpolation weights. I evaluated several λ combinations, including the assignment-required (0.3, 0.3, 0.4). Table 2 summarizes the training and dev perplexities for five different settings.

$(\lambda_1, \lambda_2, \lambda_3)$	Train PPL	Dev PPL
(0.30, 0.30, 0.40)	15.30	286.63
(0.10, 0.30, 0.60)	11.15	352.23
(0.20, 0.30, 0.50)	12.88	306.16
(0.10, 0.20, 0.70)	10.14	395.85
(0.05, 0.25, 0.70)	9.99	432.50

Table 2: Interpolated trigram perplexities on train and dev for different λ choices.

We observe a clear trade-off:

- Larger λ_3 (more weight on the trigram MLE) drives *training* perplexity down, because the model can fit long, specific contexts.
- However, too much weight on the trigram term hurts *development* perplexity due to sparsity.

Among the tested combinations, the best dev perplexity was obtained with

$$(\lambda_1, \lambda_2, \lambda_3) = (0.3, 0.3, 0.4),$$

which balances unigram, bigram, and trigram contributions more conservatively. Using this best setting, the interpolated trigram *test* perplexity was:

$$\text{PPL}_{\text{test}} \approx 286.17.$$

This is vastly better than the unsmoothed trigram MLE on dev/test (which had perplexity on the order of 10^7), confirming that interpolation substantially mitigates sparsity at evaluation time.

4.3 Effect of Using Half the Training Data

If we use only half of the training data to estimate the counts, we would expect perplexity on previously unseen data (dev/test) to increase. With fewer observations, many contexts become even sparser, leading to less reliable bigram and trigram estimates. Interpolation softens this effect but cannot fully overcome the loss of evidence: more n -grams back off to lower-order statistics and the model becomes less certain overall. In general, reducing training data increases perplexity, especially for higher-order n -grams.

4.4 Effect of a Stricter <UNK> Threshold

If we map all tokens with frequency < 5 to <UNK> (instead of < 3), the vocabulary shrinks even more and more uncommon words share the same <UNK> symbol. This changes the bias-variance trade-off.

- On one hand, the model now observes <UNK> in more contexts, which can *improve* probability estimates for rare events and slightly lower dev perplexity.
- On the other hand, we lose lexical specificity, because moderately rare but meaningful words are collapsed into <UNK>.

For a dataset of this size, increasing the <UNK> threshold from 3 to 5 typically leads to a mild improvement on dev/test perplexity for n -gram models, since the dominant problem is sparsity rather than over-generalization. However, if the threshold were pushed too high, performance would eventually degrade because the model would become overly coarse.

5 Experiments with OPT-1.3B (Part 3)

The original assignment references GPT-3, which is no longer available. Following the instructor’s update, I used the open-source `facebook/opt-1.3b` model via the HuggingFace `transformers` library. All experiments are purely prompt-based; there is no fine-tuning.

5.1 Zero-Shot Reading Comprehension

I first evaluated OPT-1.3B on short reading comprehension questions in a zero-shot setting. The prompts followed the pattern:

Read the passage and answer the question. [passage] Question: [question] Answer:

I used four passages, including the assignment’s Harvard example (Joseph Willard / Samuel Webber) and three additional questions about Marie Curie, the Apollo 11 mission, and the Amazon rainforest.

The behavior was clearly mixed:

- For the Harvard passage, the correct answer is *Samuel Webber*. OPT-1.3B instead answered *Henry Ware* and continued with fabricated commentary (e.g., unrelated discussion of Harvard and additional sentences). This illustrates a failure in multi-sentence reasoning and entity tracking.
- For the Marie Curie question, the model correctly answered that she won Nobel Prizes in Physics and Chemistry, but then continued generating extra biography and repeated parts of the prompt.

- For the Apollo 11 question, it correctly identified Michael Collins as the astronaut who stayed in orbit, again with some repetition.
- For the Amazon rainforest question, it correctly produced the nickname “lungs of the Earth” but then hallucinated a follow-up question about the Amazon river that was not requested.

Overall, OPT-1.3B can answer some well-known factual questions but is unreliable on less common or more context-dependent questions. It frequently repeats the input and produces additional, unnecessary text.

5.2 Few-Shot

Next, I tried doing few-shot prompting to see if giving the model some solved examples would help it answer the question correctly. I added a short passage about Robert Boyle and John Mayow, along with two example questions and answers. After that, I included the Harvard passage again and asked the model the same question: “Who succeeded Joseph Willard as president?”

Even with those examples, OPT-1.3B still got the answer wrong. Instead of saying Samuel Webber, it repeated the earlier incorrect answer Henry Ware, and it also made up an extra question and answer about Samuel Webber’s death year that I never asked for.

From this experiment, it seems like few-shot prompting didn’t really help the model understand the passage any better. Instead, the model mostly copied the style of the examples without actually improving its reasoning or accuracy. It felt like it was following the pattern of a Q&A format rather than actually reading and understanding the new passage.

5.3 Summarization Task

For the second task, I asked OPT-1.3B to summarize short paragraphs about NLP and machine learning. The prompt template was:

`Summarize the following paragraph in one or two sentences. [paragraph] Summary:`

In both cases, the model failed to produce genuine summaries. It largely repeated the original text and added extra commentary, without substantially compressing or abstracting the content. This indicates that, without instruction tuning or task-specific guidance, OPT-1.3B behaves more like a generic text continuation model than a robust summarizer.

5.4 Discussion

Compared to modern instruction-tuned models (such as GPT-style “Instruct” or chat models), OPT-1.3B shows several limitations:

- It frequently hallucinates additional facts and questions.
- It often repeats large portions of the prompt.
- Its zero-shot reading comprehension accuracy is inconsistent, and few-shot prompting did not consistently fix errors.
- For summarization, it tends to copy and extend the paragraph rather than produce a clear abstract.

These behaviors seem consistent with a mid-sized autoregressive language model trained to continue text, rather than a model optimized to follow instructions or answer questions reliably.

6 Conclusion

I implemented MLE unigram, bigram, and trigram language models and evaluated them on a subset of the One Billion Word Benchmark. The unsmoothed trigram model achieved very low perplexity on the training data but severely overfit, with extremely large perplexity on dev and test due to sparsity. Linear interpolation between unigram, bigram, and trigram probabilities dramatically improved dev and test perplexities, with the best performance at $(\lambda_1, \lambda_2, \lambda_3) = (0.3, 0.3, 0.4)$.

In Part 3, my experiments with OPT-1.3B showed that while the model can answer some factual questions and produce fluent text, it is prone to hallucination, repetition, and unreliable reading comprehension, even with few-shot prompting. Summarization behavior was particularly weak without additional guidance.